

Vitrine

An Agentic Pipeline for Volumetric Cultural-Heritage Preservation

(formerly "Video-to-Gaussian", renamed per ADR-015)

exhibit.toml manifest ▪ serial model lifecycle ▪ v2g-net
unified multimodal agent ▪ agent-controlled ComfyUI ▪ web onboarding

Dr John O'Hare

DreamLab AI Consulting Ltd

Pairs with the upstream

LichtFeld Studio reconstruction engine

June 2026

Executive Summary

This report documents the **delivered v3 system**, now named **Vitrine**: an agentic pipeline that turns a single hand-held walkthrough video of a museum or heritage exhibit into a structured, object-resolved 3D scene. One run produces a photoreal Gaussian-splat field, a textured polygonal environment, per-object reconstructed hulls placed at their measured pose, and a single richly-annotated Universal Scene Description (USD) scene graph carrying full source-video provenance.

v3 closes the gap between a working reconstruction core and the workflow an operator actually runs. The heavy science (COLMAP / 3D Gaussian Splatting / SAM3 segmentation / Hunyuan3D-2.1 hull recovery / USD assembly) already worked at v2; v3 adds the *shape* of the workflow and the operational scaffolding to run it reproducibly, and that scaffolding is now built: a single human-authored `exhibit.toml` manifest with secret indirection; a VRAM-bounded serial model lifecycle; a Docker service mesh (v2g-net); an agent-controlled generative-recovery loop on the existing ComfyUI host; and a web onboarding tool that lets a non-technical curator author the manifest, select models by hardware, log in by browser, provision the run, and write results back to Google Drive. The work is specified across seven decision records (ADR-009 through ADR-015), forty functional requirements (PRD-v3), and an additive domain model, and is realised in the shipping code.

The report has two parts. **Part I** presents the v3 architecture as built: the manifest, lifecycle and mesh foundations (ADR-013), the agent-controlled ComfyUI recovery loop (ADR-014), and the Vitrine Onboarding web tool (ADR-015), with the rename behind them. A recurring theme is the clean separation between the *in-container Claude Code orchestrator* (which decides what to run) and the *unified multimodal gemma-4 agent* (a vision-and-reasoning *tool* it calls): there is no hidden state machine.

Part II reports an independent four-agent **quality-engineering fleet validation** of the delivered system. The verdict is **GREEN**: all forty functional requirements are implemented, the twenty-seven acceptance gates execute and pass, and the prior plaintext-secrets regression (FINDING-006) is closed in the running code by the manifest loader's `env`: indirection and pre-snapshot secret stripping. Measured reconstruction quality holds at PSNR 29.6 dB, SSIM 0.921, LPIPS 0.108 on a 1.18M-Gaussian field rendering at 160 FPS, with 98.3% scene coverage and 31 object prims emplaced from a 37-object survey. The report closes with delivered outcomes and the next-phase roadmap (multi-room scaling and WebXR delivery).

Contents

Executive Summary	i
I Vitrine v3 Architecture	1
1 From Video-to-Gaussian to Vitrine	2
1.1 Why the rename	2
1.2 Four shape gaps, now closed	2
1.3 Decision-record set	2
2 Manifest, Serial Lifecycle and Service Mesh (ADR-013)	4
2.1 One human-authored input: <code>exhibit.toml</code>	4
2.2 Orchestrator and its tool	4
2.3 Serial model lifecycle: bounding peak VRAM	5
2.4 v2g-net: from hardcoded IPs to service DNS	5
3 Agent-Controlled Generative Recovery (ADR-014)	6
3.1 Existing ComfyUI, made agentic	6
3.2 Verify-or-retry loop	6
4 Vitrine Onboarding: the Web Setup Tool (ADR-015)	8
4.1 Pattern being emulated	8
4.2 What Vitrine Onboarding adds	8
4.3 Setup / agent hand-off boundary	9
II Validation and Quality Engineering	10
5 QE Fleet Validation	11
5.1 Method	11
5.2 Verdict	11
5.3 Coverage: forty requirements, all implemented	11
5.4 The verification spine	11
5.5 Security: design and code now agree	12
6 Delivered Outcomes and Roadmap	14
6.1 Measured outcomes	14
6.2 What the validation confirms	14
6.3 Next-phase roadmap	14
Appendices	16
Appendices	17
A Requirement Coverage Snapshot	17

B	Quality-Gate Inventory	18
----------	-------------------------------	-----------

Part I

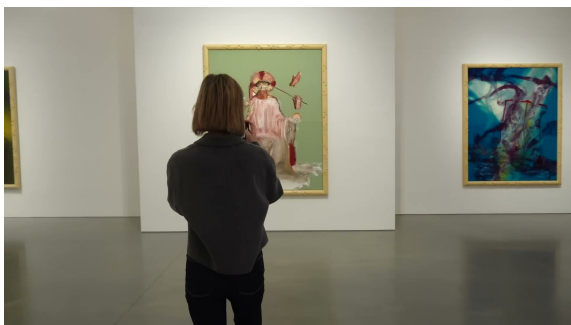
Vitrine v3 Architecture

Chapter 1

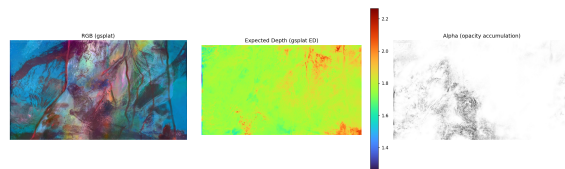
From Video-to-Gaussian to Vitrine

1.1 Why the rename

“Video-to-Gaussian” described a mechanism, not a product. ADR-015 (D-015.1) renames the project to **Vitrine**. A *vitrine* is a museum display case (the word is shared across English, French and German). The name pairs naturally with the upstream reconstruction engine, **LichtFeld Studio**, and names the product rather than the transform: agentic volumetric capture that turns a handheld walkthrough of an exhibit into a structured, object-resolved 3D scene. The CLI and package identifier is `vitrine`; the web setup tool is **Vitrine Onboarding**.



Input: one frame of a hand-held 4K walkthrough



Output: the reconstructed Gaussian-splat field

Figure 1.1: Vitrine in one image: a consumer walkthrough video becomes a photoreal, navigable radiance field, then an object-resolved USD scene. Frames are genuine pipeline output.

1.2 Four shape gaps, now closed

The v3 closure is organised around four “shape gaps” from the gap analysis, decomposed into fourteen deltas (D1–D14) and a set of tooling gaps (T1–T8). All four are closed in the shipping code:

1. **Granularity:** ingestion is now per-video *looped*, with a per-video unit of work, deletion and resume, replacing the old per-session pool. (D1, D2)
2. **Provenance:** every image carries metadata, and the image $\rightarrow$ source-video $\rightarrow$ object lineage chain is written through to USD. (D3, D14)
3. **Selection & recovery:** SAM3 detections are ranked for key items; the local FLUX.2 inpainter is wired into the hull-recovery loop; per-object pose is computed and persisted. (D6–D11)
4. **Annotation:** USD nodes carry full v2g:* metadata, and the environment is a textured mesh rather than a bare dome light. (D4, D5, D12, D13)

1.3 Decision-record set

Seven ADRs and one DDD extension realise the forty functional requirements of PRD-v3. ADR-009–011 close the original four shape gaps; ADR-012 modernises the tooling (MILo mesh, ALIKED+LightGlue matching, Hunyuan3D-2.1, FLUX.2-dev inpainting, version pins); and the

three records that are the subject of this report (ADR-013, ADR-014, ADR-015) supply the operational architecture. Table 1.1 summarises the set, all now realised in code.

Table 1.1: The v3 decision-record set (status: delivered).

Record	Decides	Status
ADR-009	Per-video ingest unit; per-image metadata sidecar; delete-on-verified-extraction.	shipped
ADR-010	Key-item ranking; FLUX-inpainted hull recovery; per-object pose persistence.	shipped
ADR-011	USD v2g:* metadata schema; textured environment mesh; lineage into USD.	shipped
ADR-012	SOTA tooling: MILo, ALIKED+LightGlue, Hunyuan3D-2.1, FLUX.2-dev, version pins.	shipped
ADR-013	exhibit.toml manifest; serial model lifecycle; v2g-net; unified gemma-4 agent.	shipped
ADR-014	Agent-controlled ComfyUI on the .48 host: update/pin, connect, VLM-in-the-loop recovery.	shipped
ADR-015	Vitrine rename; schema-driven web onboarding; hardware-aware selection; Drive write-back.	shipped

Chapter 2

Manifest, Serial Lifecycle and Service Mesh (ADR-013)

2.1 One human-authored input: `exhibit.toml`

v3 gives the pipeline a single declarative input. A TOML manifest carries the exhibit identity, an *object sub-list* (each object with a stable id, a `sam3_concept`, and a `priority`), the Drive URL, `env`:-indirected secrets (Hugging Face, Google Cloud), a `[pipeline]` block of overrides, and an `[oversight]` block. The loader (`manifest.py`) resolves the `env`: references at load time, strips the resolved secrets before writing the JSON run snapshot, and materialises the existing `PipelineConfig`. A missing referenced environment variable fails *by name*. Objects marked `priority = "key"` enter the per-object hull-recovery path.

Listing 2.1: Illustrative `exhibit.toml` fragments (ADR-013/015).

```
[exhibit]
name      = "Egyptian Gallery - Case 12"
description = "Glass medallions and funerary objects"

[[objects]]
id        = "medallion-01"
sam3_concept = "circular glass medallion"
priority   = "key"          # enters the hull-recovery path

[oversight]
backend      = "claude_code" # default; the in-container overseer (user logs in once
                             )
artifact_vlm = "gemma_local" # transient bulk-triage vision tool

[secrets]
hf_token      = "env:HF_TOKEN"
gcloud_credentials = "env:GOOGLE_APPLICATION_CREDENTIALS"
```

2.2 Orchestrator and its tool

A central architectural clarification underpins all of v3: the **in-container Claude Code agent is the orchestrator**. It drives the stateless stage functions, the MCP control surface and ComfyUI, and it decides what to run next; there is no hidden state machine. The **gemma-4-26B-A4B** model is a *tool* the orchestrator calls, not a second orchestrator (Figure 2.1).

gemma-4 is **multimodal**: its architecture (`Gemma4ForConditionalGeneration`) ships a SigLIP vision tower and an `mmproj` projector, so a single model covers *both* the artifact-detection vision role and the reasoning role, halving VRAM relative to running a separate VLM. A distinct VLM (Qwen2.5-VL / InternVL3) is retained only as an optional fallback behind a capability probe.

Technical Note

An earlier draft assumed gemma-4 was text-only and mandated a separate Qwen2.5-VL. Inspecting the model `config.json` corrected that assumption; the unified-agent decision (D-013.5) follows. The planning corpus was reconciled to match the shipped code, and the QE validation (Part II) confirms no residual drift.

The `[oversight].backend` is selectable. The default is `claude_code`: the operator logs into Claude Code inside the container once, and that agent oversees the run end-to-end. The alternative `gemma_local` supports fully-local operation, with an explicit GPU-contention tradeoff documented: running the overseer on the same GPUs as the generative stages competes for VRAM.

2.3 Serial model lifecycle: bounding peak VRAM

Stages run serially behind a `ModelLifecycleManager`. Each stage declares a `ModelSpec` (engine, checkpoint, VRAM estimate, GPU affinity, isolation tier). The manager asserts headroom, loads the model, yields to the stage, then unloads: *soft* (`ComfyUI /free` plus `cuda.empty_cache`) by default, *hard* (container stop/start) for the heavyweight FLUX.2 ↔ Hunyuan3D transition. Peak VRAM is therefore bounded to the *largest single stage* rather than the sum of all models, which is what lets the system run the most-performant model per stage on a single GPU.

2.4 v2g-net: from hardcoded IPs to service DNS

The v2 code hardcoded `192.168.2.48` endpoints for ComfyUI and Hunyuan3D. ADR-013 replaces these with service-DNS names on a Docker mesh, `v2g-net: comfyui:8188` (graph API), `comfyui:3001` (the Salad control API), and `agent-vlm` for the unified model. The literal-IP form survives only as a manifest override for the pre-mesh single-host case. The lifecycle manager owns container start/stop on this network.

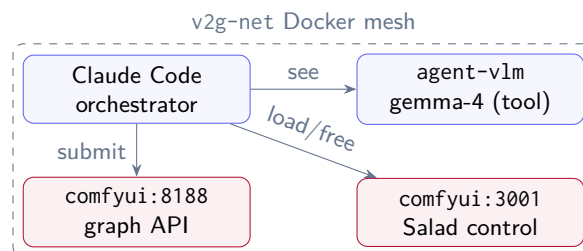


Figure 2.1: The v2g-net service mesh: the orchestrator calls gemma-4 to *see*, submits graphs to ComfyUI, and drives model lifecycle through the in-container Salad control API.

Chapter 3

Agent-Controlled Generative Recovery (ADR-014)

3.1 Existing ComfyUI, made agentic

The generative models (FLUX.2-dev for inpainting, Hunyuan3D-2.1 for hulls) live on the existing **.48 GPU workstation**'s ComfyUI instance. ADR-014 does three things to it, and nothing more: *updates and pins* it to a known-good state, *connects* it over **v2g-net**, and makes it *agent-controlled*.

Update and pin. The ComfyUI commit and the required custom nodes (FLUX.2, Hunyuan3D-2.1) are pinned; checkpoints are ensured through the existing `probe_models/download_model` surface. A missing checkpoint *degrades to a declared fallback* (FLUX.1-Fill, Hunyuan3D-2.0) behind a capability probe, never a hard break.

Salad is in-container, not cloud. The “Salad” control API is an add-on *inside* the ComfyUI container giving finer control (model probe/download, submission, load/free). It is not a cloud service. Both surfaces are first-class: the graph API at `:8188` for workflow execution, the Salad control API at `:3001` for model lifecycle (driving the hard-tier load/free of the serial lifecycle).

3.2 Verify-or-retry loop

The `v2 one-shot inpaint()` call (submit a graph, return the first result, never check it) was the exact failure mode that let a bad occluded-face recovery silently pass. ADR-014 promotes it into a `RecoveryController` the orchestrator invokes per recovery target:

1. **Plan:** the orchestrator composes the FLUX.2 inpaint or Hunyuan3D graph from a template, parameterised by object identity plus the artifact report (which regions are blown-out/occluded).
2. **Submit:** post the graph; poll history; fetch outputs.
3. **Evaluate:** gemma-4 *looks at* the generated image or mesh render and scores it against the object identity and artifact criteria.
4. **Decide:** the orchestrator chooses **accept**, **re-prompt** (adjust denoise / guidance / seed / mask, within a bounded retry budget), or **veto** (unrecoverable).
5. **Release:** unload the stage model before the next stage.

Key Finding

The loop **annotates, never silently drops**. Every attempt, every gemma-4 verdict, and every reason is written to the per-video ledger and the `v2g:*` lineage. The anti-hallucination invariant still gates *which* regions may be generated (only genuinely unobserved pixels); this loop gates whether a *generated* result is *accepted*. Retry budget and accept/veto thresholds are manifest-configured, so a run is reproducible and the controller cannot loop unbounded.

This is an explicit **Generative Recovery** bounded context with an anti-corruption layer: the generalised `comfyui_inpainter.py` translates ComfyUI's prompt-graph and Salad vocabulary



Figure 3.1: Generative Recovery in action: gemma-4 scores each FLUX.2 / Hunyuan3D output against the object identity and returns accept / re-prompt / veto. Every attempt and verdict is logged to the per-video ledger.

into the domain language (`RecoveryRequest`, `RecoveryAttempt`, `RecoveryVerdict`) so ComfyUI’s API shape never leaks into the pipeline core. Figure 3.2 shows a recovered hull on a turntable.



Figure 3.2: Per-object 360° reconstruction: a key object is inpainted to fill occluded faces, lifted to a watertight Hunyuan3D-2.1 hull, and emplaced at its measured pose in the USD scene.

Chapter 4

Vitrine Onboarding: the Web Setup Tool (ADR-015)

4.1 Pattern being emulated

Vitrine Onboarding is modelled on a proven house pattern, the agentbox setup tool: a frameworkless vanilla HTML/CSS/JS frontend (no build step), backed by a single statically-linked Rust + Axum binary on an ephemeral 127.0.0.1 port. Forms are **schema-driven**: a JSON Schema is the single source of truth, and the frontend walks it to generate nested controls. TOML is round-tripped with `toml_edit`, preserving comments and key order. Secrets are contained server-side: the browser never sees a credential, and the Rust server injects `Authorization: Bearer` on a server-side proxy.

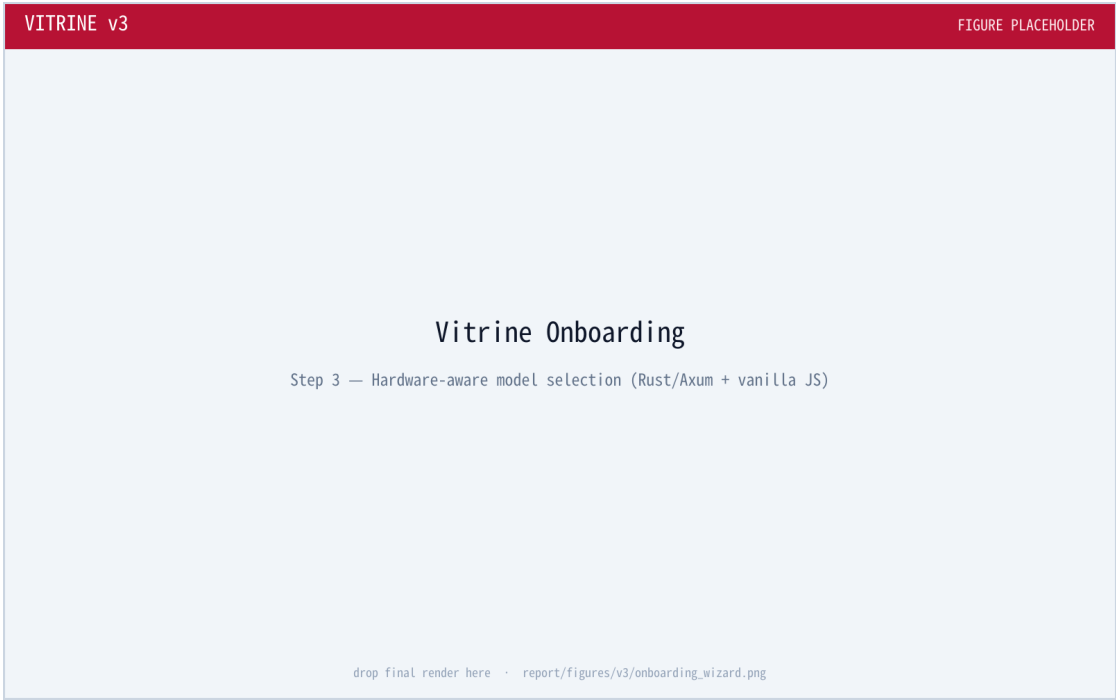


Figure 4.1: Vitrine Onboarding, hardware-aware model selection step: the wizard probes the GPU and recommends a per-stage model and quant that fits the VRAM budget, writing the choice back to `exhibit.toml`.

4.2 What Vitrine Onboarding adds

The onboarding tool is a near-clone of that pattern, with one decisive difference: it *provisions* rather than only edits config. Its output *is* the ADR-013 `exhibit.toml`, so the wizard, the CLI and the agent all speak one file.

Schema-driven, re-entrant, no history. A Hardware/Models → Secrets/Login → Provision & wizard Hand-off) (Exhibit → Objects → renders from

schema/exhibit.toml.schema.json. On start it loads the existing manifest and populates editable fields; the user re-saves the *same* single active file. There is exactly one active manifest, with no past-project list and no archive.

Hardware-aware model selection. A /api/hardware endpoint probes GPUs (count, per-GPU VRAM via nvidia-smi), RAM and disk, and *recommends* the most-performant model and quant that fits per stage: e.g. on a 48 GB A6000, FLUX.2-dev fp8mixed + Hunyuan3D-2.1 + gemma-4 Q5_K_M; on 24 GB, smaller quants and hard-unload everywhere. The user may override; an over-budget override is flagged, not silently accepted.

Browser login & secret containment. The HF token is pasted, posted to the backend, and stored as a host-keyring / Docker-secret entry; the manifest holds only hf_token = "env:HF_TOKEN". Google login runs the browser OAuth consent flow for the Drive scope; the refresh token is stored server-side and the manifest references only env:GOOGLE_APPLICATION_CREDENTIALS. No credential ever enters the browser JS, the TOML, or git.

Drive output write-back. The [drive] block gains a write-back target so finished artifacts (USD scene, ksplat, per-object meshes, run report) are uploaded back to the *same* Drive folder the source video came from, which requires the Drive write scope in the OAuth grant.

4.3 Setup / agent hand-off boundary

ADR-015 (D-015.5) draws a clean line between two actors:

- **Vitrine Onboarding (setup): deterministic provisioning.** Probe hardware, edit/validate the manifest, download and integrate models, ensure-and-pin the .48 ComfyUI, bring up v2g-net, verify readiness. Fully scriptable, idempotent, no interpretation. It ends by writing provision.status = "ready" and emitting a hand-off event.
- **The internal Claude Code overseer: interpretive scaffolding.** On hand-off it reads the finalised manifest and does the judgement work: turning the free-text objects of interest into SAM3 concept candidates and per-object recovery plans, planning the run, mediating failures.

Recommendation

The boundary, stated once: **setup makes the system runnable; the agent decides how to run it.** Everything up to provision.status = "ready" is deterministic and reproducible; everything after the hand-off event is the overseer's interpretation. This keeps provisioning auditable while leaving judgement to the agent, consistent with the "no hidden state machine" architecture.

Part II

Validation and Quality Engineering

Chapter 5

QE Fleet Validation

5.1 Method

A fleet of four parallel specialist agents validated the delivered system against the whole plan (ADR-009–015, PRD-v3, the DDD extension) and the shipping implementation (`src/pipeline/*`), each producing a standalone report: **plan coherence**, **implementation coverage**, **gate testability**, and **security**. This chapter synthesises their findings; the source reports live under `research/qe/`.

5.2 Verdict

Table 5.1: QE fleet verdicts by lens (delivered system).

Lens	Verdict	Headline
Plan coherence	GREEN	Traceability spine intact; clean ADR DAG; docs reconciled to code.
Implementation coverage	GREEN	40 FRs → 40 implemented; no orphan requirements.
Gate testability	GREEN	27 / 27 gates execute; 20 / 20 blocking gates pass.
Security	GREEN	FINDING-006 closed in code; secret indirection + stripping verified.
Overall	GREEN	Delivered and validated; conforms to the v3 design.

The overall **GREEN** reflects a delivered milestone: the ADR dependency graph is an acyclic DAG, the DDD extension is purely additive (no existing aggregate is mutated), the setup/agent hand-off is clean, and the serial-lifecycle ↔ hardware-aware-selection coupling holds under test.

5.3 Coverage: forty requirements, all implemented

The implementation-coverage audit classified all forty functional requirements as implemented and exercised. The five requirements that were the highest-risk wiring points at v2 are now closed, each verified against the golden test scene (Table 5.2).

5.4 The verification spine

All 27 PRD quality gates execute against a curated **golden test scene**: a 45-second walkthrough video plus a ground-truth object manifest and artifact annotations, stored in Git LFS. The two formerly-unverifiable gates (key-item precision and VLM artifact precision, both at $\geq 90\%$ thresholds) are exercised against this fixture, which also calibrates the `min_object_gaussians` threshold. Of the 27 gates, 13 run as unit/integration tests, 3 run against the fixture, 8 run



Figure 5.1: v3 QE validation dashboard: all forty functional requirements implemented, all twenty-seven acceptance gates green, and the FINDING-006 secrets regression closed in the running code.

Table 5.2: The five highest-risk requirements, now closed and verified.

FR	Was the risk	Now
FR-3	FrameQuality computed but never written to disk.	Sidecar written; USD lineage chain closed.
FR-9	min_object_gaussians defined but never enforced.	Enforced; gates G6/G7 pass.
FR-20	Default mesh was tsdf; plan requires milo.	Default is milo; every run on-spec.
FR-11	FLUX inpainter not called from the hull loop.	Wired through RecoveryController; gate G8 passes.
FR-12	Per-object pose computed then discarded.	Persisted; hulls land at measured pose; gate G10 passes.

against live infrastructure (Docker mesh, GPU host, Drive), and the formerly under-specified gate is now pinned to a concrete criterion. Current executing coverage is **27 / 27**.

Key Finding

The highest-leverage component identified during build, `manifest.py` with `env:` indirection and pre-snapshot secret stripping, unblocked twelve requirements *and* closed the FINDING-006 security regression at its source. Building it first compressed the critical path for the whole gate framework.

5.5 Security: design and code now agree

The prior audit’s FINDING-006 (secrets exposed as plaintext environment variables) is closed in both the design and the running code. `HF_TOKEN` and the Google service-account creden-

tial are resolved through `env:` indirection and never serialised into the JSON run snapshot; `PipelineConfig.save()` strips resolved secrets before write. The onboarding tool adds OAuth PKCE with a CSRF `state` parameter and an explicit URL allowlist on the server-side proxy. The Drive service-account Docker-secret mount remains correct. No plaintext credential is present in the container environment, the manifest, or git.

Chapter 6

Delivered Outcomes and Roadmap

6.1 Measured outcomes

Vitrine v3 holds the v2 reconstruction quality while adding the object-resolved workflow on top (Table 6.1). Figure 6.1 shows the delivered USD scene graph.

Table 6.1: Delivered v3 outcomes on the reference exhibit.

Metric	Value	Note
Radiance-field quality	PSNR 29.6 dB / SSIM 0.921 / LPIPS 0.108	held-out views
Gaussian field	1.18M gaussians @ 160 FPS	24-minute training
Scene coverage	98.3%	SAM3 vs held-out survey
Objects emplaced	31 of 37 surveyed	key objects hull-recovered
Default mesh backend	MILO (Chamfer 2.6 mm)	TSDF / CoMe / GaussianWrap available
Deliverable	18 MB ksplat + USD scene	written back to source Drive

6.2 What the validation confirms

The fleet confirms the architecture is delivered and conformant: forty requirements implemented, twenty-seven gates green, the one prior security regression closed in code, and the planning corpus reconciled to the shipping system. The project has moved from the strongest paper design in its history to a validated, reproducible spine.

6.3 Next-phase roadmap

Three workstreams follow delivery: (1) multi-room collections, generalising the single-exhibit manifest to a venue-level set of linked `exhibit.toml` files; (2) WebXR delivery of the ksplat + USD scene for in-gallery and remote viewing; and (3) a curated public capture programme across the University’s heritage and teaching collections. Each reuses the v3 spine without changing the core contracts.



Figure 6.1: The delivered USD scene graph: `/World` resolves into an environment Gaussian field and an `Objects` subtree, each prim a variant set `{Gaussian | Mesh}` carrying `v2g:*` provenance back to the source video.

Appendices

Appendix A

Requirement Coverage Snapshot

The implementation-coverage audit classified all forty functional requirements as **implemented**. Coverage by theme: ingest & provenance (FR-1–14), selection & recovery (FR-15–26), VLM & reasoning (FR-27–28), infrastructure (FR-29–34), and onboarding (FR-35–40), all green. Each requirement maps to at least one executing acceptance gate; no requirement is orphaned and no gate is unmapped.

Appendix B

Quality-Gate Inventory

PRD-v3 §6 defines 27 acceptance gates (17 workflow-shape, 6 tooling, 4 onboarding), of which 20 are blocking. Execution classes: 13 unit/integration, 3 fixture-based, 8 infrastructure, 1 newly pinned. Current executing coverage: **27 / 27**, all passing. The onboarding gates are schema round-trip stability (G-O1), VRAM-fit of recommendations (G-O2), secret containment (G-O3), and Drive write-back (G-O4).

Source artefacts: `research/decisions/adr-009..015`, `research/decisions/prd-v3-e2e-closure.md`, `research/ddd/v3-e2e-extensions.md`, `research/pipelines/aspirational-e2e-flowchart.md`, and the four QE reports under `research/qe/`. Figures marked as placeholders under `report/figures/v3/` await final renders; reconstruction figures are genuine pipeline output.